

```
from sklearn.preprocessing import StandardScaler
scaler=StandardScaler()
df[["age_std","fare_std"]]=scaler.fit_transform(df[["age","fare"]])
df[["age_std","fare_std"]].head()
```

	age_std	fare_std
0	-0.530377	-0.502445
1	0.571831	0.786845
2	-0.254825	-0.488854
3	0.365167	0.420730
4	0.365167	-0.486337

```
from sklearn.preprocessing import RobustScaler
scaler=StandardScaler()
df[["age_robust","fare_robust"]]=scaler.fit_transform(df[["age","fare"]])
df[["age_robust","fare_robust"]].head()
```

	age_robust	fare_robust
0	-0.530377	-0.502445
1	0.571831	0.786845
2	-0.254825	-0.488854
3	0.365167	0.420730
4	0.365167	-0.486337

Day 8 EDA

An Outlier is a data point that is significant different from the rest of the data

Example:

Passenger fares:

7.25

8.05

10.50

512.33

outlier is 512.33 because this is not normal as remaining data why detect outliers?

outlier can

- * Distort the mean
- * Affect standard deviation
- * reduce model performance
- * influence scaling and regression models

Method 1: Visual Detection using box plot

A box plot quickly shows

1. Minimum value
2. Q1 (25th percentile)
3. Median

4. Q3(75th percentile)
5. Maximum value
6. Outliers

Method 2: IQR Method

IQR=Interquartile Range

Formula : $IQR = Q3 - Q1$

Step 1 : Calculate IQR

step 2 : Calculate limits-lower limit & -upper limit

step 3 : Detect outliers

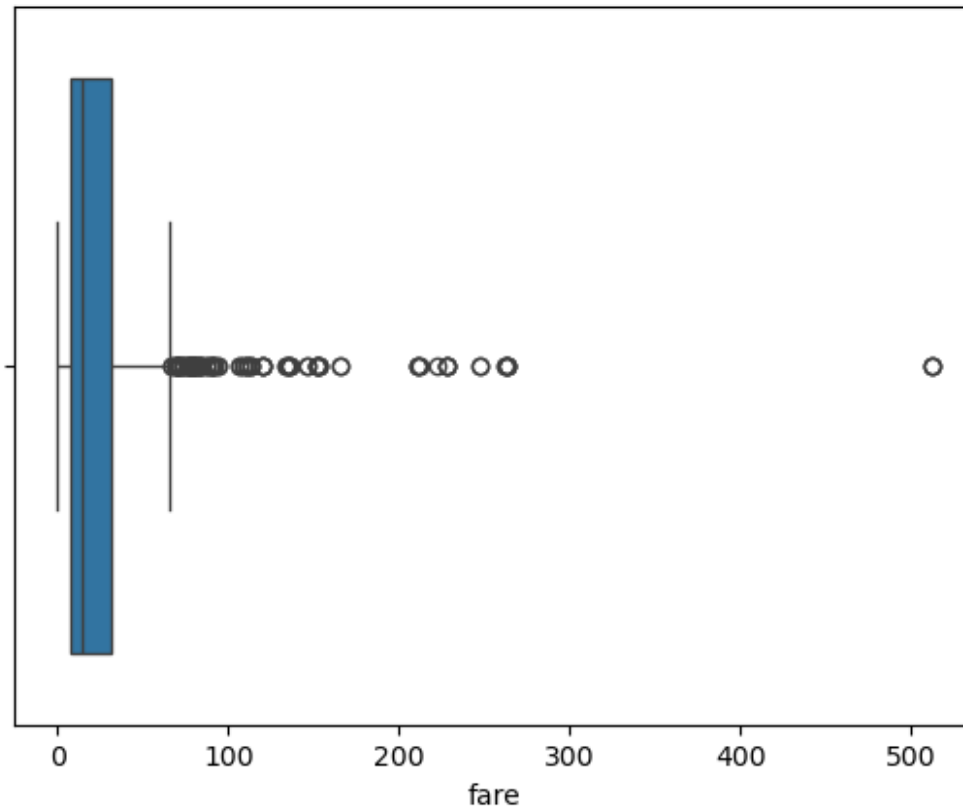
Method 3 : Z-Score Method

Measures how many standard deviations a value is from the mean

Step 1 : Calculate Z-Score

step 2 : Detect outliers

```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from scipy.stats import zscore
#Load Titanic Dataset
df=sns.load_dataset("titanic")
#Box plot
sns.boxplot(x=df["fare"])
plt.show()
#IQR Method
Q1=df["fare"].quantile(0.25)
Q3=df["fare"].quantile(0.75)
IQR=Q3-Q1
lower=Q1-1.5*IQR
upper=Q3+1.5*IQR
outliers_iqr=df[(df["fare"]<lower) | (df["fare"]>upper)]
#print("Outliers: ",outliers_iqr)
print("IQR Outliers: ",len(outliers_iqr))
# Z-Score Method
df["fare_zscore"]=zscore(df["fare"])
outliers_z=df[abs(df["fare_zscore"])>3]
print("Z-score Outliers: ",len(outliers_z))
```



IQR Outliers: 116
Z-score Outliers: 20

EDA(Exploratory data analysis) **is** the process of examining, summarizing, **and** visualizing data before building machine learning models

Why EDA **is** important?

Understand the structure of the dataset

Identify missing values

Detect outliers

Understand feature distributions

Find relationships between variables(correlation)

Check data quality issues

Prepare data **for** machine learning

Types of EDA

1. Univariate Analysis

Analysis one variable at a time

Numerical features:

Mean, Median, mode, standard deviation, variance, histogram, box plot

Categorical Features:

Value Counts, Frequency Distribution, Bar chart

2. Bivariate Analysis

3. Multivariate Analysis

```

import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
df=sns.load_dataset("tips")
print(df.head())
print(df.info())
print(df.columns)

```

	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

```

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 244 entries, 0 to 243
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   total_bill  244 non-null    float64
1   tip         244 non-null    float64
2   sex         244 non-null    category
3   smoker      244 non-null    category
4   day         244 non-null    category
5   time        244 non-null    category
6   size        244 non-null    int64
dtypes: category(4), float64(2), int64(1)
memory usage: 7.4 KB
None
Index(['total_bill', 'tip', 'sex', 'smoker', 'day', 'time', 'size'],
      dtype='object')

```

1.Descriptive Statistics

```

print(df["total_bill"].describe())

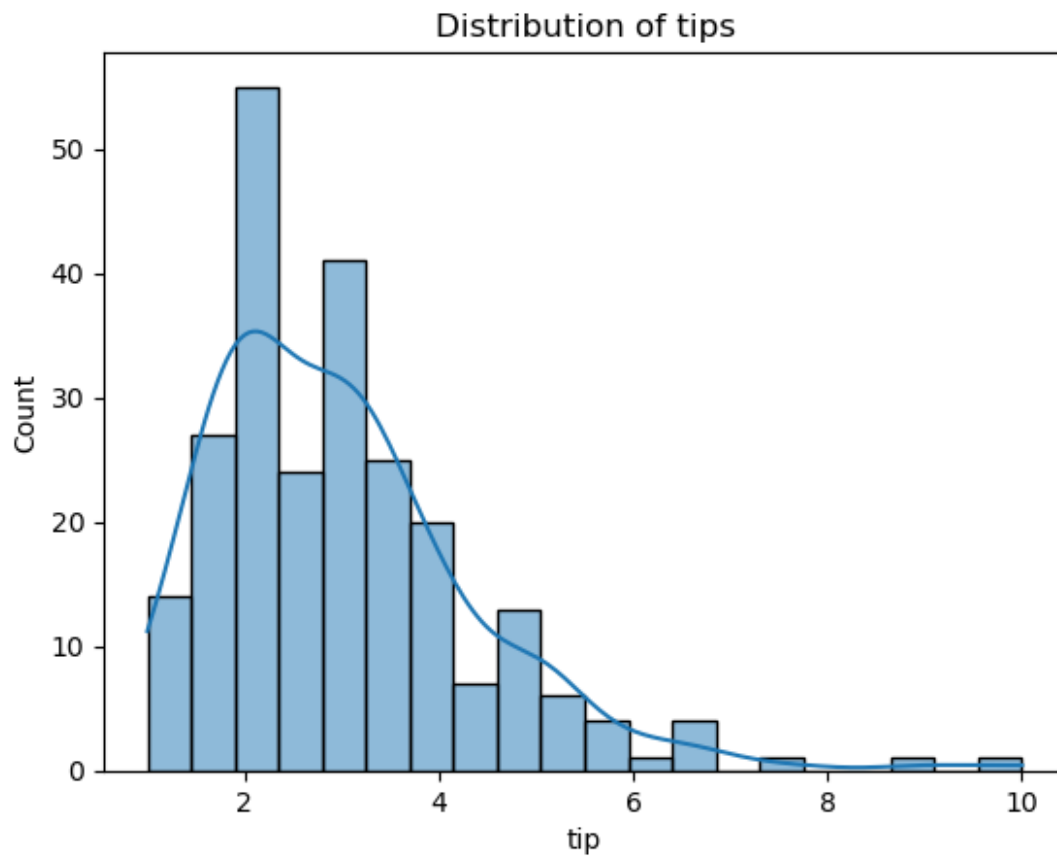
```

count	244.000000
mean	19.785943
std	8.902412
min	3.070000
25%	13.347500
50%	17.795000
75%	24.127500
max	50.810000

Name: total_bill, dtype: float64

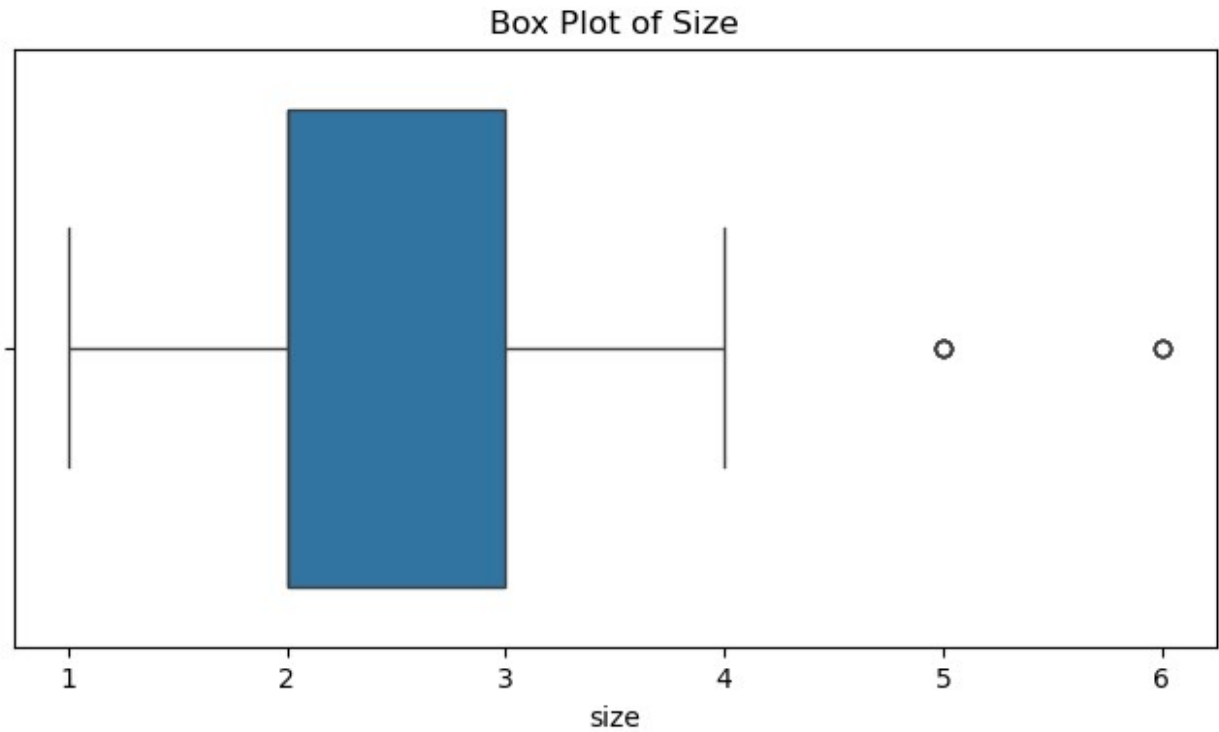
1. Histogram
 - A histogram shows the distribution of a numerical feature
 - It divides data into intervals(bins) and shows how many observations fall into each interval

```
sns.histplot(df["tip"],bins=20,kde=True)
plt.title("Distribution of tips")
plt.show()
```



```
#Box Plot
Box plot summarizes

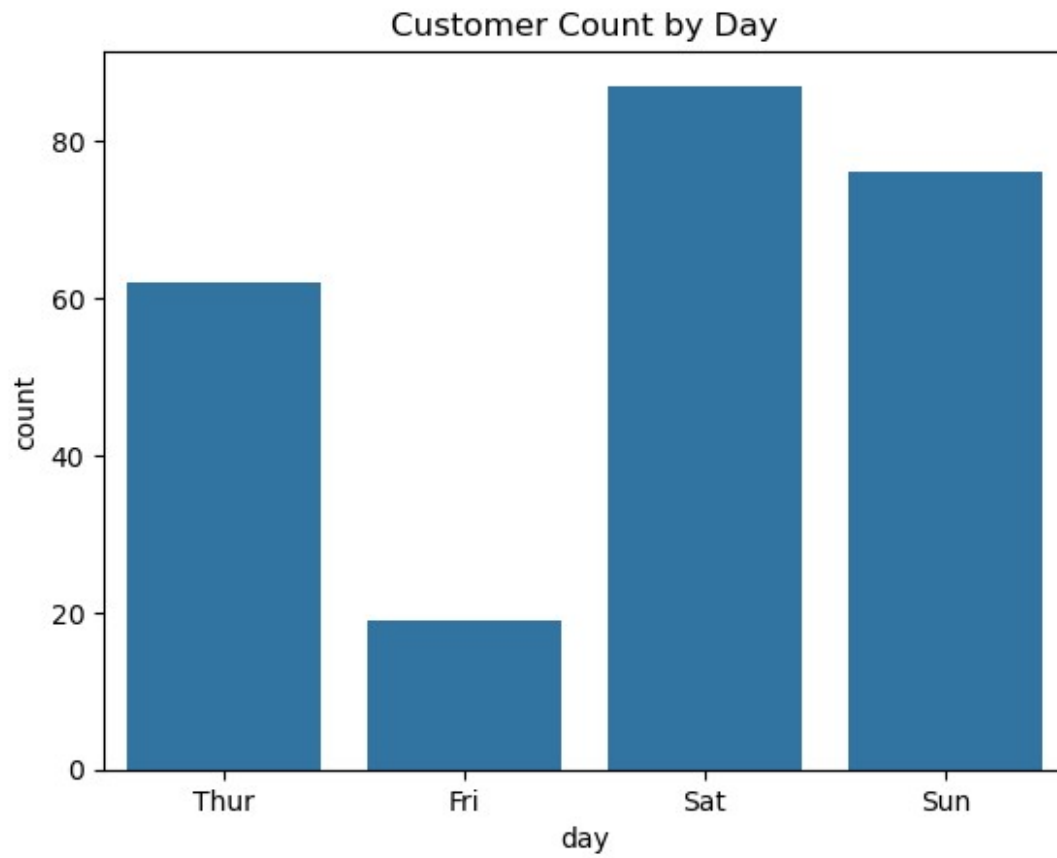
plt.figure(figsize=(8,4))
sns.boxplot(x=df["size"])
plt.title("Box Plot of Size")
plt.show()
```



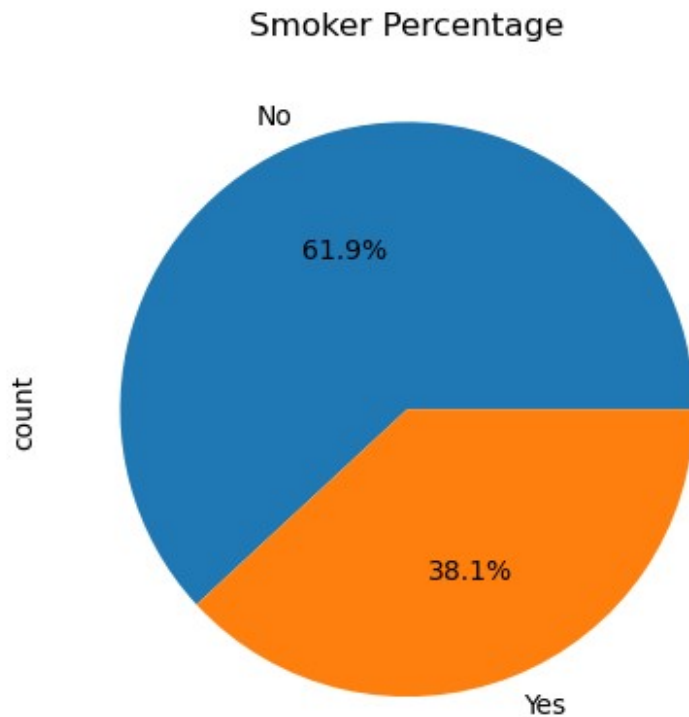
```
#2.Categorical Variable : Day
#Value Counts
print(df["day"].value_counts())

day
Sat      87
Sun      76
Thur     62
Fri      19
Name: count, dtype: int64

#count plot
sns.countplot(x="day",data=df)
plt.title("Customer Count by Day")
plt.show()
```



```
#Pie Chart
df["smoker"].value_counts().plot(
    kind="pie",
    autopct="%1.1f%%"
)
plt.title("Smoker Percentage")
plt.xlabel("")
plt.show()
```



```
# Outlier Detection Using IQR
Q1=df["total_bill"].quantile(0.25)
Q3=df["total_bill"].quantile(0.75)
IQR=Q3-Q1
lower=Q1-1.5*IQR
upper=Q3+1.5*IQR
outliers=df[(df["total_bill"]<lower) | (df["total_bill"]>upper)]
print("Outliers: ",len(outliers))
#print("IQR Outliers: ",len(outliers_iqr))
# Z-Score Method
#df["fare_zscore"]=zscore(df["fare"])
#outliers_z=df[abs(df["fare_zscore"])>3]
#print("Z-score Outliers: ",len(outliers_z))

Outliers: 9
```

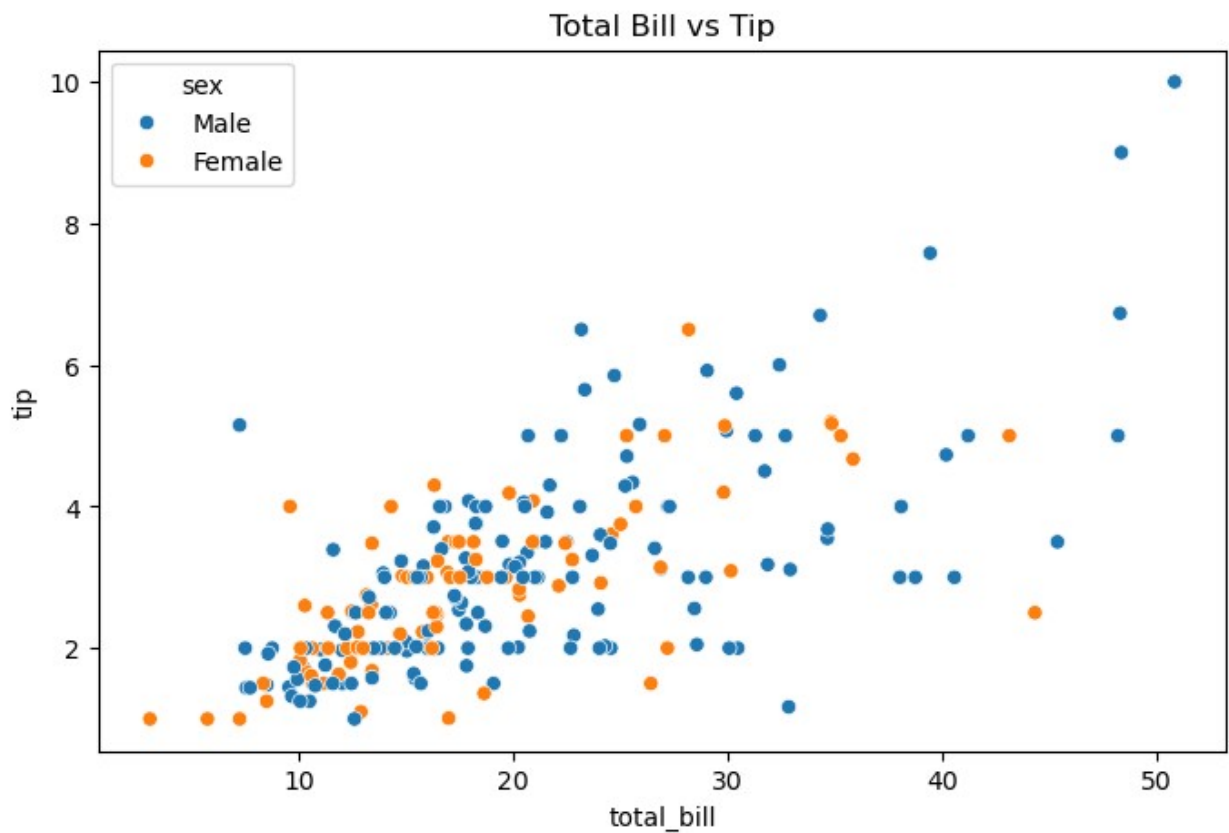
Bivariate Analysis means analyzing two variables together to understand the relationship between them

Bi=Two Variant=Variables Examples:

- Study Hours vs Exam Score
- Age vs Salary
- total bill vs Tip
- Types of Bivariant Analysis | Variable 1 | Variable 2 | visualization |
 |-----|-----|-----| | Numerical | Numerical | Scatter
 Plot | | Categorical | Numerical | Bar chart | | Categorical | Categorical | Grouped
 statistics/Crosstab |

Numerical vs numerical

```
plt.figure(figsize=(8,5))
sns.scatterplot(
    x='total_bill',
    y='tip',
    hue='sex',
    data=df
)
plt.title("Total Bill vs Tip")
plt.show()
```



```
# Correlation
# Pearson correlation
```

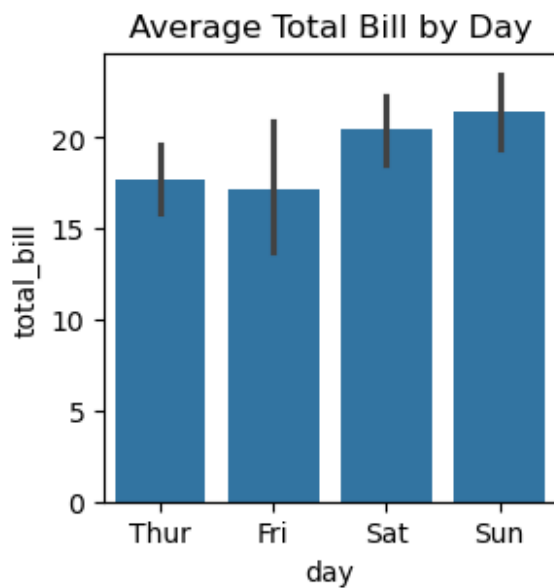
```
corr=df["total_bill"].corr(df["tip"])
print(corr)

0.6757341092113641
```

Categorical vs numerical

Days vs Total bills

```
# bar chart
plt.figure(figsize=(3,3))
sns.barplot(
    x="day",
    y="total_bill",
    data=df
)
plt.title("Average Total Bill by Day")
plt.show()
```



```
# Grouped Statistics
# Mean bill by Day
# To avoid va
print(df.groupby("day", observed=False)["total_bill"].mean())
#multiple Statistics
print(df.groupby("day", observed=False)
      ["total_bill"].agg(["mean", "median", "min", "max"]))
```

```

day
Thur    17.682742
Fri     17.151579
Sat     20.441379
Sun     21.410000
Name: total_bill, dtype: float64

```

	mean	median	min	max
day				
Thur	17.682742	16.20	7.51	43.11
Fri	17.151579	15.38	5.75	40.17
Sat	20.441379	18.24	3.07	50.81
Sun	21.410000	19.63	7.25	48.17

```

C:\Users\devad\AppData\Local\Temp\ipykernel_11688\2109595883.py:4:
FutureWarning: The default of observed=False is deprecated and will be
changed to True in a future version of pandas. Pass observed=False to
retain current behavior or observed=True to adopt the future default
and silence this warning.

```

```

    print(df.groupby("day")["total_bill"].mean())
C:\Users\devad\AppData\Local\Temp\ipykernel_11688\2109595883.py:6:
FutureWarning: The default of observed=False is deprecated and will be
changed to True in a future version of pandas. Pass observed=False to
retain current behavior or observed=True to adopt the future default
and silence this warning.
    print(df.groupby("day")
["total_bill"].agg(["mean", "median", "min", "max"]))

```

categorical vs categorical

sex vs Smoker Compares frequencies between categories

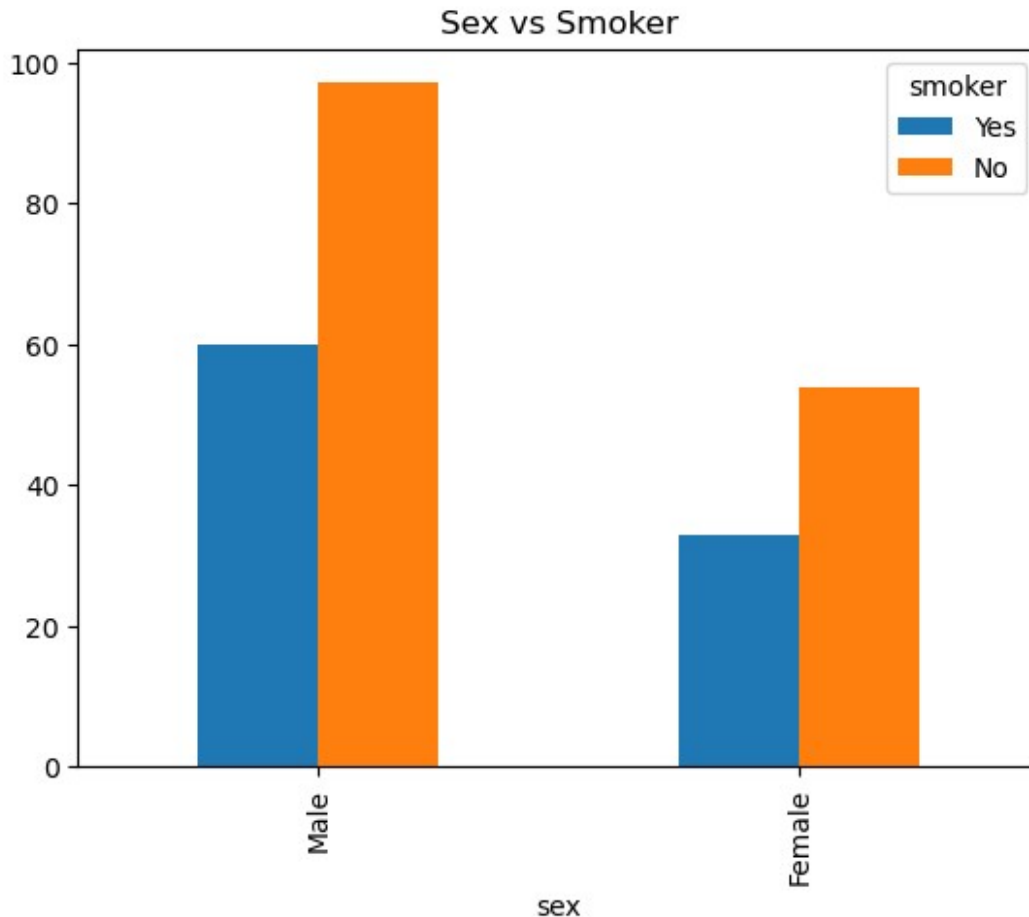
Crosstab

crosstab is used to analyse the relationship between two categorical variables

```

# bar chart
pd.crosstab(
    df["sex"],
    df["smoker"]).plot(kind="bar")
plt.title("Sex vs Smoker")
plt.show()

```



Multivariant Analysis

Multivariant Analysis means analyzing three or more variables simultaneously to understand complex relationships and patterns in data

Example:

Instead of Studying:

1. Height alone (univariate)
2. Height vs Weight (bivariate)

We study :

Height vs Weight vs Age vs Gender

This is multivariate analysis

objectives of multivariate analysis

1. Discover hidden relationship
2. understand interactions among multiple features
3. Detect clustering and patterns
4. Support feature selection
5. Prepare data for machine Learning

1. Pairplot

A pairplot compares every numerical feature with every other numerical

feature

It automatically creates:

- 1.Scatter plot
- 2.Histograms
- 3.Relationship matrix

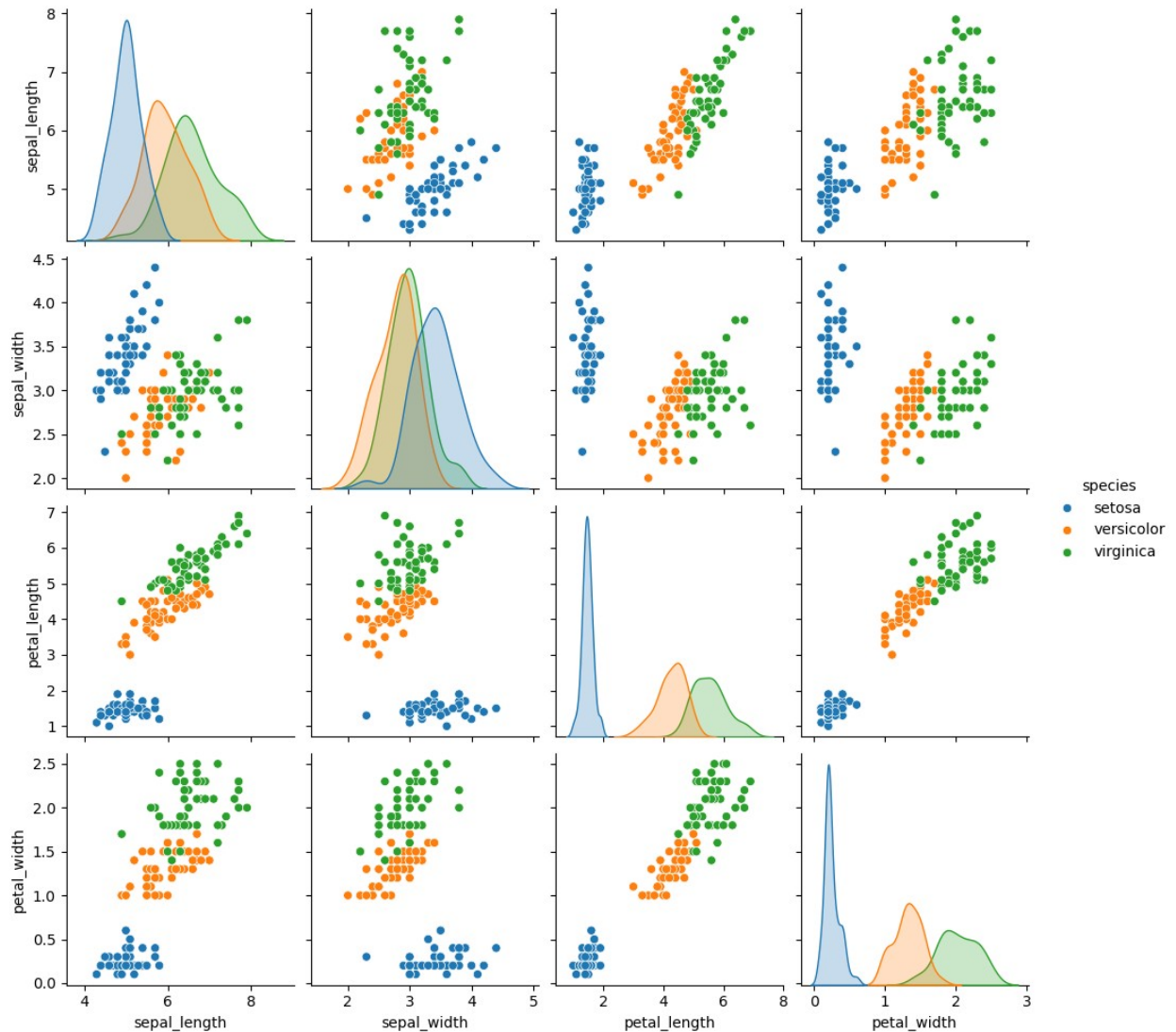
Why use pairplot??

- 1.Detect correlations
- 2.Find clusters
- 3

#pairplot

```
import seaborn as sns
import matplotlib.pyplot as plt
df=sns.load_dataset("iris")
print(df.columns)
sns.pairplot(df,hue="species")
plt.show()
```

```
Index(['sepal_length', 'sepal_width', 'petal_length', 'petal_width',
      'species'],
      dtype='object')
```



2. Color-Coded Scatter Plot(3rd dimension)

1. X axis -> variable 1

2. Y axis -> variable 2

A third variable can be represented using colors
example:

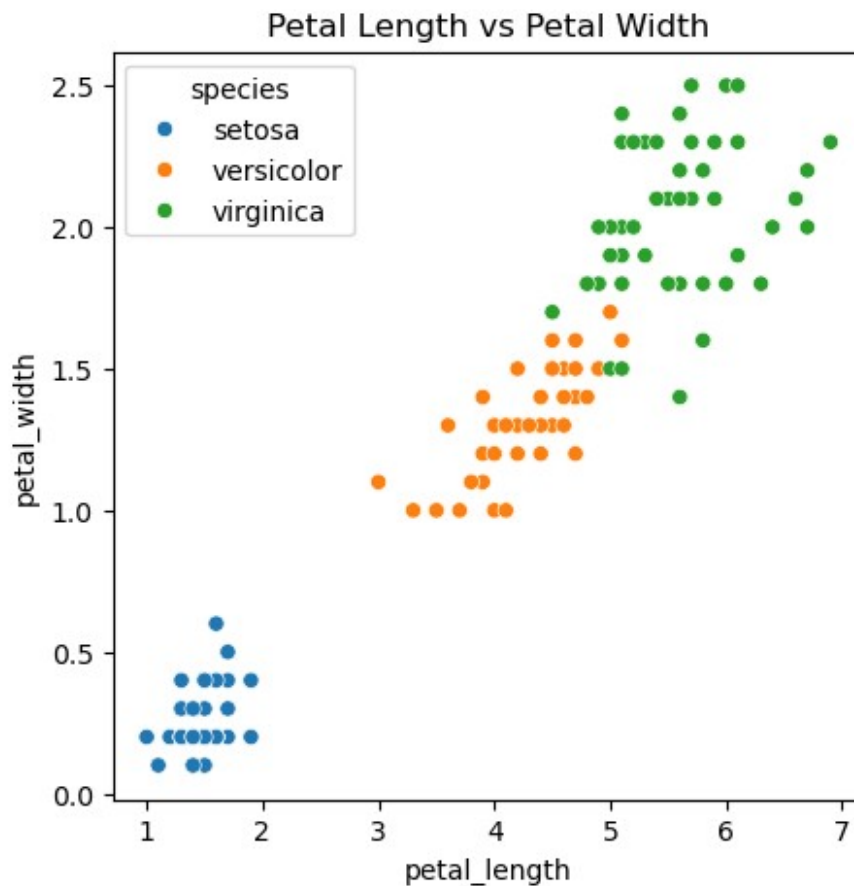
1. X -> Petal Length

2. Y -> Petal width

3. Color -> Species

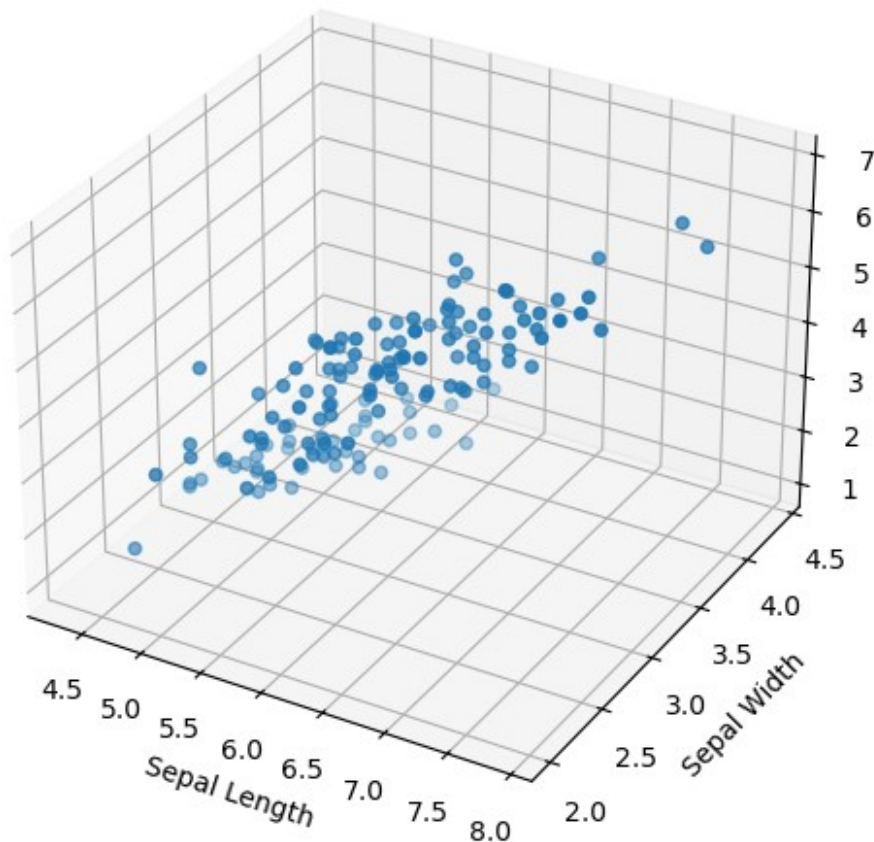
```
plt.figure(figsize=(5,5))
sns.scatterplot(
    x="petal_length",
    y="petal_width",
    hue="species",
    data=df)
```

```
plt.title("Petal Length vs Petal Width")
plt.show()
```



```
# 3D Scatter Plot
A 3D Scatter Plot uses:
1. X axis
2. Y axis

from mpl_toolkits.mplot3d import Axes3D
import matplotlib.pyplot as plt
fig=plt.figure(figsize=(8,6))
ax=fig.add_subplot(111,projection="3d")
ax.scatter(
    df["sepal_length"],
    df["sepal_width"],
    df["petal_length"])
ax.set_xlabel("Sepal Length")
ax.set_ylabel("Sepal Width")
ax.set_zlabel("petal Length")
plt.show()
```



Correlation Analysis

correlation measures the strength and direction of relationship between two numerical variables

correlation range:

$-1 \leq r \leq 1$

```
corr_matrix=df.corr(numeric_only=True)
```

```
print(corr_matrix)
```

	sepal_length	sepal_width	petal_length	petal_width
sepal_length	1.000000	-0.117570	0.871754	0.817941
sepal_width	-0.117570	1.000000	-0.428440	-0.366126
petal_length	0.871754	-0.428440	1.000000	0.962865
petal_width	0.817941	-0.366126	0.962865	1.000000

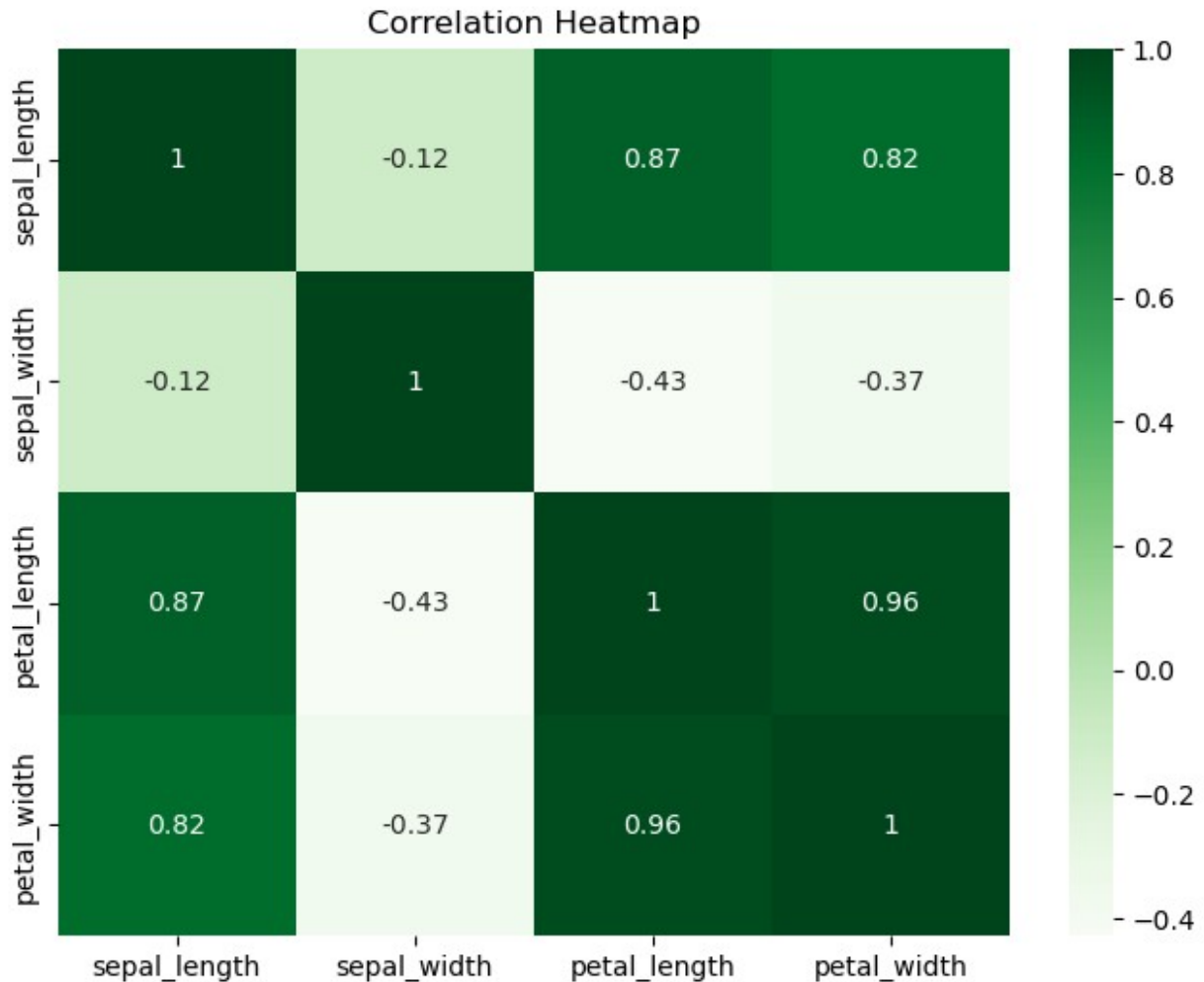
```
plt.figure(figsize=(8,6))
```

```
sns.heatmap(df.corr(numeric_only=True),
            annot=True,
            cmap="Greens"
```

```
)
```

```
plt.title("Correlation Heatmap")
```

```
plt.show()
```

VIF(Variance Inflation Factor) for multicollinearity Detection
 Multicollinearity occurs when independent variables(features) are highly correlated with each other

Example

* Suppose a house price dataset contains:

1. House Area(sq.ft)
2. Number of rooms

* these features may contain similar information and be highly correlated

* This creates multicollinearity

VIF Interpretation Table

VIF	Interpretation
1	No Correlation
1-5	Low Correlation
5-10	moderate multicollinearity
>10	severe multicollinearity

```

import pandas as pd
from statsmodels.stats.outliers_influence import
variance_inflation_factor
X=df.drop("species",axis=1)
vif=pd.DataFrame()
vif["Feature"]=X.columns
vif["VIF"]=[variance_inflation_factor(X.values,i)
            for i in range(X.shape[1])]
print(vif)

```

	Feature	VIF
0	sepal_length	262.969348
1	sepal_width	96.353292
2	petal_length	172.960962
3	petal_width	55.502060

```

#remove a VIF feature
#suppose we remove sepal_length
X_new=X.drop("sepal_length",axis=1)
vif_after=pd.DataFrame()
vif_after["Feature"]=X_new.columns
vif_after["VIF"]=[variance_inflation_factor(X_new.values,i)
                  for i in range(X_new.shape[1])]
print(vif_after)

```

	Feature	VIF
0	sepal_width	5.856965
1	petal_length	62.071308
2	petal_width	43.292574

```

#Heatmap After Removing Feature
plt.figure(figsize=(8,6))
sns.heatmap(X_new.corr(),
            annot=True,
            cmap="coolwarm"
            )
plt.title("After VIF Treatment")
plt.show()

```

